

Algorithms for Data Science
Exam 2025
Group A

June 2025

Your Name: _____ Your Student ID: _____

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	5	4	4	6	5	9	5	4	4	5	5	4	0	60
Score:														

The last task gives 4 bonus points. Thus, you can obtain 60+4 points in total (regular+bonus points).

Check if your name and student ID are printed on top of every single page!

Write your name and your student ID on the first page!

Check whether you received all 14 pages (containing 13 tasks in total).

The regular time to solve the exam is 90 minutes.

1. 5 points Consider the following incomplete Python function. The input is an array $a[0..n-1]$ and a target value x . The array is sorted reversely, that is, $a[0]$ is the largest value in a , and $a[n-1]$ the smallest one. The function outputs the maximum position i of the array such that all values in a from position 0 up to position i have value at least x . In other words, the function returns the maximum i such that, for all $0 \leq j \leq i$, we have $a[j] \geq x$. If $a[0]$ is smaller than x , then the function should return -1 .

Hint: the task is related to the first programming task.

```

1  def compute_max_pos(a, x):
2      n = len(a)
3      left = -1
4      right = n - 1
5      # ??? [Line 5] (Starts with while...)
6      # ??? [Line 6]
7      # ??? [Line 7] (Starts with if...)
8      # ??? [Line 8]
9      else:
10     # ??? [Line 10]
11
12     return left

```

Complete the code by choosing a correct code snippet for each empty line (that starts with # ???). Do it by filling the blanks with the correct labels from the code snippets below:

- | | |
|---------------------------------------|--|
| • Line 5: <u> G </u> | • Line 8: <u> I </u> |
| • Line 6: <u> O </u> | |
| • Line 7: <u> A </u> | • Line 10: <u> C </u> |

Code snippets to choose from:

- | | |
|---|--|
| • (A) <code>if x > a[p]:</code> | • (I) <code>right = p - 1</code> |
| • (B) <code>p = right - left</code> | • (J) <code>if x < a[p]:</code> |
| • (C) <code>left = p</code> | • (K) <code>if x == a[p]:</code> |
| • (D) <code>left = p + 1</code> | • (L) <code>p = (left + right + 1) * 2</code> |
| • (E) <code>right = p</code> | • (M) <code>while left == right:</code> |
| • (F) <code>right = p + 1</code> | • (N) <code>left = p - 1</code> |
| • (G) <code>while left < right:</code> | • (O) <code>p = (left + right + 1) // 2</code> |
| • (H) <code>while left > right:</code> | |

Note: The indentation of your code is the same as of # ???. In other words, your code for an empty line will start at the position of #.

2. 4 points Write a recurrence for the running time of the following algorithm `AlgoA(a[1..n])` in the form of

$$T(n) = aT(n/b) + \Theta(f(n)) .$$

You don't need to solve the recurrence! And you don't need to understand what the algorithm computes!

The input of `AlgoA(a[1..n])` is an array a of n integers.

The input of the auxiliary function `Copy_Sub_Array(a[1..n], i, m)` is an array $a[1..n]$, a position i and a length m . The function returns a copy of the subarray $a[i+1..i+m]$.

<pre> AlgoA(a[1..n]) if n ≥ 10 then ret = 0 p = n · n · n · n for j = 1 to p · (n · n) do ret = ret + a[10] d = 3 · 3 m = ⌈n/d⌉ i = n while i > n - 9 do c = Copy_Sub_Array(a, i, m) ret = ret + AlgoA(c) i = i - 1 for i = 1 to 5 do c = Copy_Sub_Array(a, i, m) ret = ret + AlgoA(c) return ret else return 7 </pre>	<pre> Copy_Sub_Array(a[1..n], i, m) b[1..m] = new empty array for j = 1 to m do b[j] = a[i + j] return b </pre>
---	---

Your answer: $T(n) = \underline{\hspace{2cm} 14T(\lceil n/9 \rceil) + \Theta(n^6) \hspace{2cm}}$

3. 4 points Solve the following recurrence using the master theorem. Determine (for yourself) the values for a , b , and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = 64 \cdot T(n/2) + \Theta(n^6)$$

Your answers:

$$c = \underline{\mathbf{6}}$$

Case: **B**

Resulting running time $T(n) = \underline{\hspace{2cm} \Theta(n^6 \log n) \hspace{2cm}}$

4. Simplify the following running times as far as possible:

(a) 2 points $\Theta(9(\log_2 n)^3 + 7\log_2(n^5) + 2) = \Theta(\underline{\hspace{2cm} (\log_2 n)^3 \hspace{2cm}})$

(b) 2 points $\Theta(9\sqrt{n} + (n^{3/6+3/6})/7 + 1) = \Theta(\underline{\hspace{2cm} n \hspace{2cm}})$

(c) 2 points $\Theta(22 \cdot n^6 + 36 \cdot 2^n + 3 \cdot 4^n) = \Theta(\underline{\hspace{2cm} 4^n \hspace{2cm}})$

5. 5 points Consider the following array $a[1..21]$ containing twenty-one elements:

$$a = \langle 105, 90, 100, 60, 85, 95, 40, 65, 25, 80, 45, 95, 75, 35, 30, 50, 55, 20, 15, 65, 70 \rangle$$

1. Draw the *complete* array as a complete binary tree (as defined for heaps),
2. Mark the edge whose endpoints (parent and child) violate the heap invariant,
3. Choose an appropriate index position j and **decrease** the value of $a[j]$ by the **maximum amount possible** such that a satisfies the heap invariant.

What index j do you choose?

(Write a single number between 1 and 21; there is only one correct answer.)

Your answer: $j =$ 8 (the violating edge's other endpoint is at 4.)

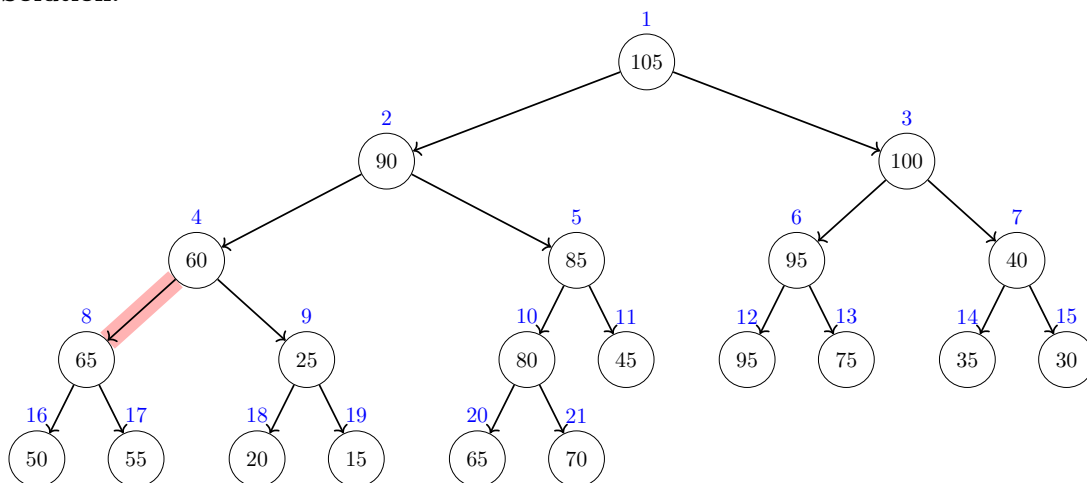
What is the new value for $a[j]$?

(Write a single number, as small as possible and in particular smaller than the old value of $a[j]$.)

Your answer: $a[j] =$ 55 (the smallest value from $[55, 60]$)

Your tree should contain **all** the elements of the array, from $a[1] = 105$ up to $a[21] = 70$:

Solution:



Explanation (not required): The heap invariant is violated at the red edge as the child (index 8) has a larger value than its parent (index 4). To fix the heap invariant, it suffices to decrease $a[8]$ to any value between 55 and 60. The smallest possible value (as asked in the task) is thus 55.

6. 9 points Dijkstra's algorithm (see below) is run on the following graph where the start node is s .

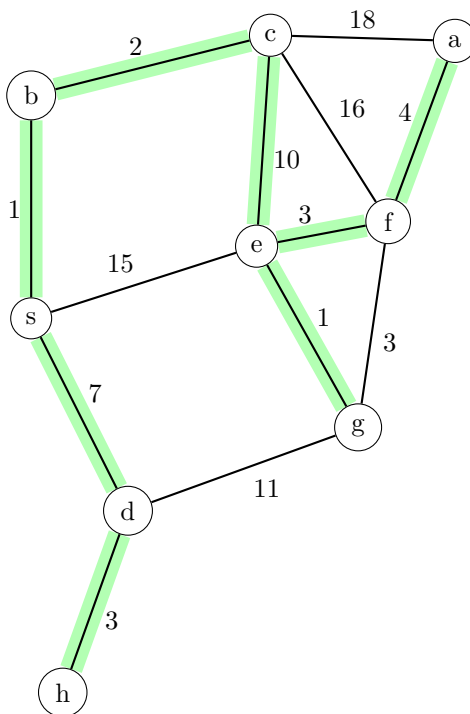
What are the values of the *dist* and *parent* tables when Dijkstra's algorithm has finished? Fill in the missing entries in the table below!

Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```

dist[0..n-1] filled with INF
parent[0..n-1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time then
        for every edge (v,w) do
            if dist[w] > dist[v] + cv,w then
                dist[w] = dist[v] + cv,w
                parent[w] = v
                q.add(w, -dist[w])

```



Solution:

	a	b	c	d	e	f	g	h	s
distance	20	1	3	7	13	16	14	10	0
parent	f	s	b	s	c	e	e	d	NONE

Note:

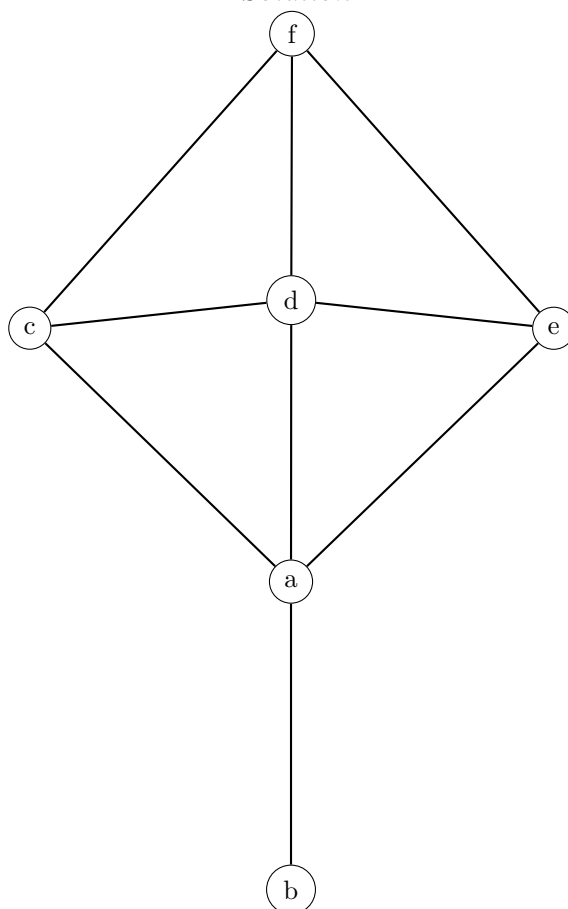
- The entries of the parent table consist of the names of the nodes or the special value *NONE*. For example, the parent of some node could be s whereas the parent of s is *NONE*.
- If a vertex has more neighbors, we consider them **in lexicographic order** (that is, node a would come before node b and so on)!

7. 5 points Draw an (undirected) graph without edge crossings corresponding to the following adjacency matrix. For your drawing, just connect the given vertices below. Your edges don't need to be straight and are allowed to be curves.

For your score, it's better to draw *all* of the edges even if they cross each other than not to draw one or more edges.

	a	b	c	d	e	f
a	0	1	1	1	1	0
b	1	0	0	0	0	0
c	1	0	0	1	0	1
d	1	0	1	0	1	1
e	1	0	0	1	0	1
f	0	0	1	1	1	0

Solution:



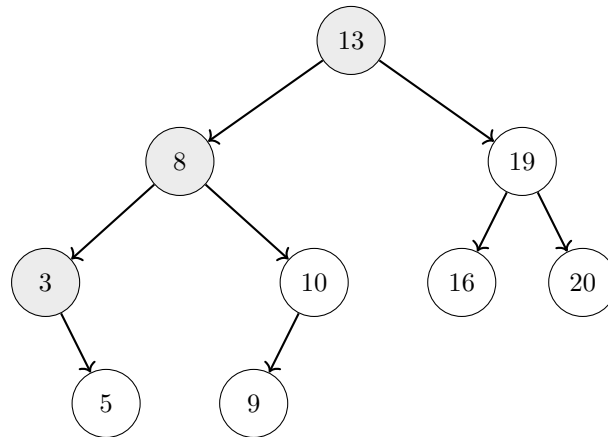
Note that a correct solution has to be crossing free! However, the depicted solution has been automatically drawn by a graph drawing program that does not guarantee that edges do not cross. Thus, if the depicted solution contains edge crossings, keep in mind that such a solution is not fully correct and you would not get all the points by drawing it. However, note that a crossing-free solution can be easily obtained from the depicted solution by redrawing one or more edges as curves around the graph.

8. 4 points Add the following keys into the following binary search tree **in the order** as they are given below. It suffices that you draw how the binary search looks at the end with each key written in its respective node. (For simplicity, we consider only keys and no values.)

Note: Do not change connections between previously added nodes! Just add the keys one by one in the given order to their correct positions.

10, 5, 9, 19, 16, 20

Solution:



9. 4 points What is the most appropriate data structure for the following scenario, chosen from the list below?

Unless stated otherwise in the scenario, the goal is to minimize memory usage and the worst-case running time of each operation.

In your answer, briefly explain how your chosen data structure applies to the scenario. Then give its memory and time complexity, and explain why it performs better than *every* other option, considering the scenario's specific constraints and goals.

You are developing a system for a small embedded device that needs to store sensor readings, each identified by a unique integer key within a small, predefined range. It is critical that inserting sensor data into your system, as well as accessing these readings by key, is performed with guarantees of efficient performance even in the worst case. Due to hardware constraints, the data structure must be very simple. Furthermore, it has only a single consecutive block of memory at its disposal for storing the keys.

1. Array indexed by keys
2. AVL tree
3. Find-union
4. Hash table
5. Heap/Priority queue
6. Sorted Linked list
7. Queue
8. Stack
9. Sorted array
10. Unsorted array

Most appropriate data structure: Array indexed by keys

Explanation:

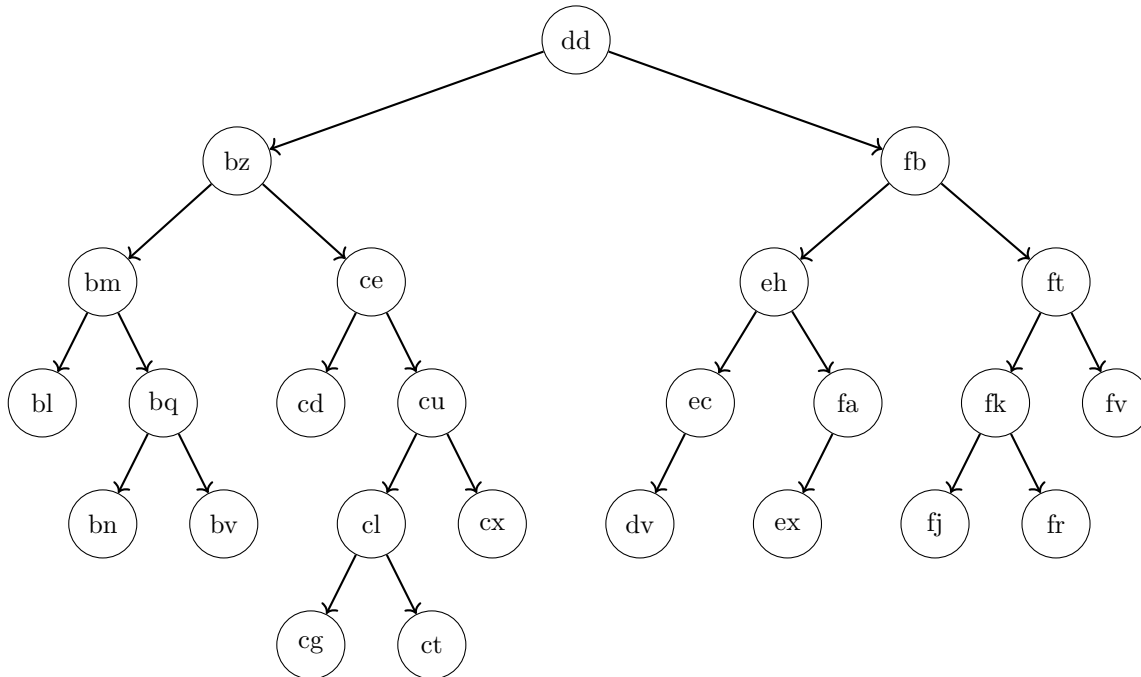
Solution: An array indexed by keys is optimal in this scenario because it allows $O(1)$ time for all standard dictionary operations for accessing the data whereas its memory usage of $O(M)$ (where M is the maximum key) is not too large as the keys belong to a small range. All the other data structures are worse under this scenario as they either don't provide feasible operations (find-union, stack, queue, heap), or take more time in the worst case (AVL tree, hash table, sorted and unsorted array, sorted linked list), while their memory complexities are not better. Furthermore, this data structure satisfies the requirement of a simple, contiguous block of memory.

10. 5 points What is the height of the node *bz* in this binary search tree, what is the height of the root (as defined in the lecture and exercises)?

Recall that the height of a node v is the number of nodes on a longest path starting at v within the subtree of v (not counting the dummy “NONE” nodes that have height 0).

Height of *bz*: 5 (Write only a single number.)

Height of the root: 6 (Write only a single number.)



Does the tree satisfy the AVL invariant? Your answer: no (Write “yes” or “no”.)

Write the names of two nodes,

$X =$ ce and $Y =$ cd, such that the following holds:

- If the tree violates the AVL invariant, then the violation occurs at node X (caused by the heights of X 's children), and adding a new child to node Y will restore the AVL invariant.
- Otherwise, if the tree satisfies the AVL invariant, then adding a new child to node X will cause a violation of the AVL invariant, and the violation will occur at node Y (caused by the heights of Y 's children).
- If there is more than one valid choice for X , choose the one with the alphabetically **smallest** name. **After selecting X** , if there is more than one valid choice for Y , choose the one with the alphabetically **smallest** name. (Example: “*bl*” is alphabetically smaller than “*fv*”.)

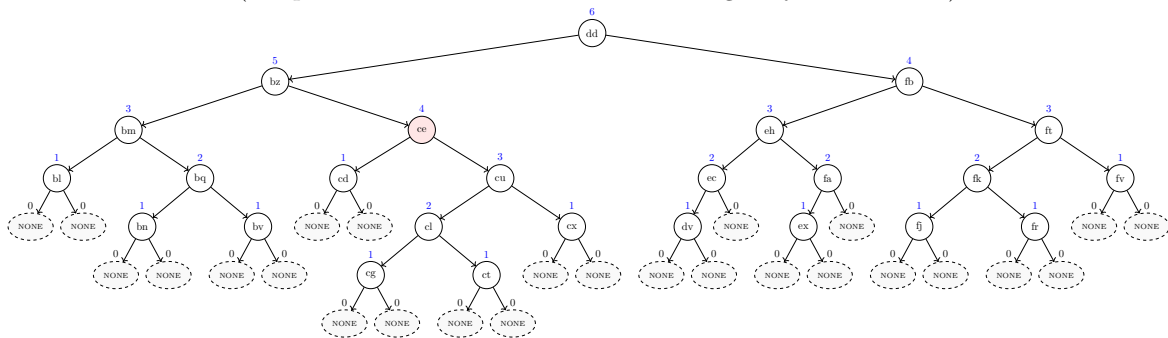
In other words, X is either a node that violates the AVL invariant, or a node where adding a new child would cause such a violation. Similarly, Y is either a node where adding a new child restores the AVL invariant, or it is a node that violates the invariant after a new child is added to X . If there are multiple valid choices for X , choose the one with the alphabetically smallest name. If there are multiple valid choices for Y (given your choice of X), choose the one with the alphabetically smallest name.

Note that there is exactly one correct solution that satisfies all the conditions above. It is also possible that $X = Y$.

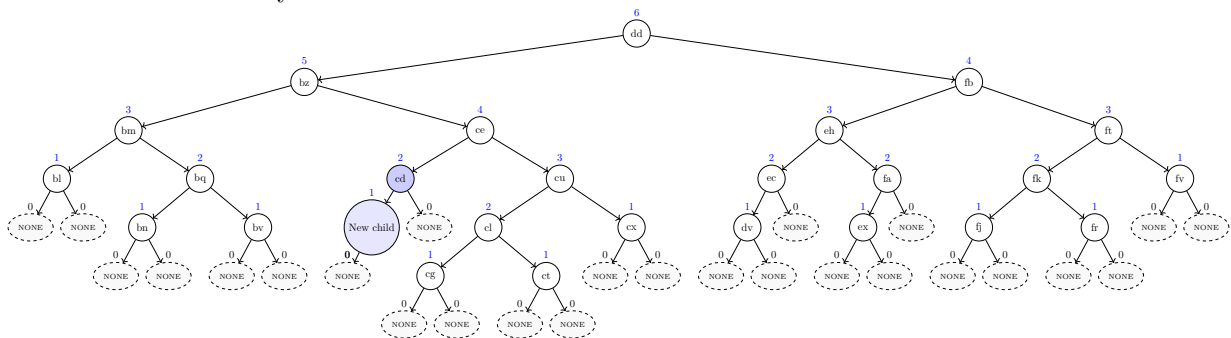
See next page for an explanation of the solution!

Explanation:

The tree violates the AVL invariant at $X = ce$. The first tree shows the heights and dummy nodes, as well as the violation in red (the parent X whose children differ in height by more than 1).



The second tree shows the parent $Y = cd$ and its new added child (both in blue) where the new child causes the tree to satisfy the AVL invariant.

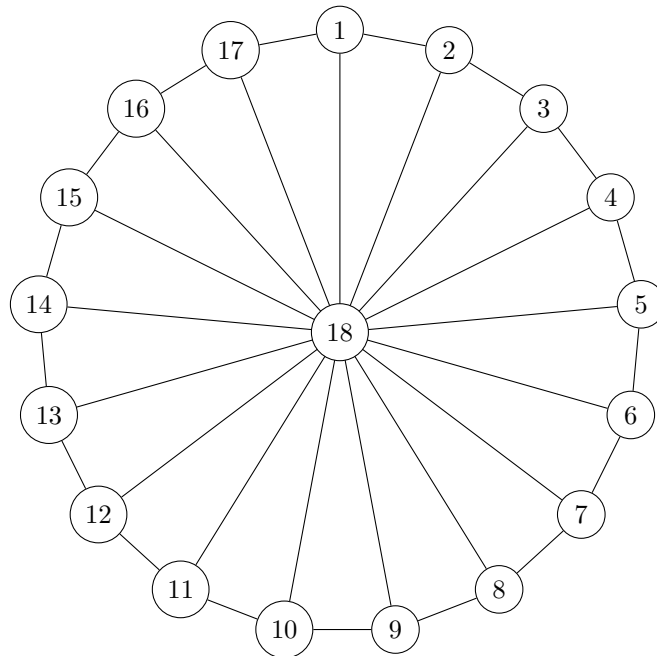


11. 5 points Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to $n=18$ and each edge has the same weight 1. Observe that the vertices 1 to 17 lie on a big cycle and each of them is connected to the center vertex 18 via an edge.

Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 7) already after 13 iterations.

```

1  $\delta[1..18][1..18] = \text{new 2D array filled with } +\infty$ 
2 for  $i = 1$  to 18 do
3    $\delta[i][i] = 0$ 
4 foreach  $\text{edge } (v, w, c)$  do
5    $\delta[v][w] = c$ 
6    $\delta[w][v] = c$ 
7 for  $k = 1$  to 13 do
8   foreach  $\text{pair } (v, w)$  do
9      $\delta[v][w] = \min(\delta[v][w], \delta[v][k] + \delta[k][w])$ 
10 return  $\delta$ 
```



Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 13 times**!

Recall that the weight of every edge is equal to 1.

If the distance is **infinite**, write "INF". Note: There are exactly two fields where the correct answer is "INF". If you write "INF" in more than two fields, you get 0 points for those fields.

$$\delta[15][3] = \underline{\text{INF}} \qquad \delta[18][15] = \underline{1}$$

$$\delta[17][14] = \underline{14} \qquad \delta[13][16] = \underline{\text{INF}}$$

$$\delta[2][12] = \underline{10}$$

12. 4 points Consider the following algorithm.

Input: integer b

$x = 4$

$e = 0$

while $x < b$ **do**

$t = 4$
 $x = x + t$
 $e = e + t$
 $x = x - 1$
 $e = e - 11$

Someone wrote the following invariant for the loop:

$$A \cdot x + C \cdot e = D$$

with the three constants

- $A = 7$
- $C = 2$
- $D = 28$

The invariant should hold each time the algorithm evaluates the loop condition. Unfortunately, the invariant has a mistake: one of the three constants, A , C , or D has a wrong value.

Fix the invariant by changing the value of one of the constants!

Which constant do you choose? _____ **C** _____ (*Write the name of one of the three constants.*)

The new value for the chosen constant: _____ **3** _____

Note: You are allowed to change only one constant.

13. 4 points (bonus)

Consider the following decision problem A:

You are given a list of users of a social network. Furthermore, you also know for every two users whether they are friends or not. Is there an ordering of some of the users (not necessarily all) such that the first user is Alice and the last user is Bob and any two consecutive users in the ordering are friends with each other?

Consider the following decision problem B:

With the exam just around the corner, you decided to watch lecture recordings. Each recording has a different length and a different rating based on how helpful it is. Unfortunately, time is short, and you won't be able to watch everything. Your goal is to maximize the total rating of the recordings you watch, but you need to carefully choose which ones to fit into your limited time. Thus, is there a selection of recordings that you can still watch in your remaining study time and whose total rating is above some given rating value?

Mark all the correct sentences and give a short explanation.

- ☒ **A can be reduced to B in polynomial time.**
- ☐ B can be reduced to A in polynomial time.
- ☒ **A is in P.**
- ☐ B is in P.
- ☐ It is unknown whether A can be reduced to B in polynomial time.
- ☒ **It is unknown whether B can be reduced to A in polynomial time.**
- ☐ It is unknown whether A is in P.
- ☒ **It is unknown whether B is in P.**

Solution: Problem B is equivalent to the NP-complete problem KNAPSACK (the items are the recordings, total capacity is the time left, and the target total value is the given rating value), whereas problem A can be solved in polynomial time: we can reduce the problem (in polynomial time) to the connectivity problem in graphs as follows: users are vertices, we have an edge between two vertices if the respective users are friends. We ask: is there a path connecting the vertex of *Alice* with the vertex of *Bob*? If there is such a path, then, going along the path, we get an ordering of users where consecutive users are friends (edges) and where the first user is *Alice* and the last one is *Bob*. On the other hand, if there is such an ordering, then there is also such a path. This completes the polynomial-time reduction.

Next, observe that the connectivity problem can be solved in polynomial time, for example, by using breadth-first search (BFS). Consequently, the problem that we reduced to the graph problem is also in P.

Consequently, as problem A can be solved in polynomial time, it can be reduced in polynomial time to any other problem, thus also to B.

Since it is unknown whether NP-complete problems can be solved in polynomial time, we do not know whether B is in P, and, consequently, whether there exist a polynomial time reduction from B to any polynomial-time problem (e.g., to A).